

Monolithic Architecture vs Microservices :

What is Software Architecture ?

Before understanding monolithic architecture and microservices , let's first understand software architecture.

It defines :

- a. How different components of an application are organised
- b. How these components communicate with each other
- c. How data flows between components
- d. How the application is deployed
- e. How the application handles growth and failures

Example : E commerce application

Imagine you want to build an e-commerce application similar to Amazon. Your application needs several features:

1. User signUp , login
2. Product catalog
3. Product search
4. Shopping cart
5. Order management
6. Payment processing
7. Delivery tracking
8. Notifications

All these features must work together to provide a complete shopping experience. But how should we organise their code?

There are different approaches.

1. **Monolithic architecture:** Put all the application's functionality into one application.
2. **Microservices architecture :** Divide the application into multiple smaller applications, each responsible for a particular functionality.

These are two different ways of organizing and deploying software.

MONOLITHIC ARCHITECTURE :

Monolithic architecture is a software architecture in which all the major components of an application are developed, packaged, and deployed as a single unit.

The word *monolithic* comes from the idea of something made from a single, large piece.

They may be organized into separate modules and layers, but they are usually deployed together.

Characteristics of Monolithic Architecture

1. **Single Codebase :** A codebase is the collection of source code files used to build an application. In a monolithic architecture, all the major application modules are usually maintained in one codebase.

The backend might contain:

```
backend/  
|  
├─ users/  
├─ products/  
├─ cart/  
├─ orders/  
├─ payments/  
└─ notifications/
```

All these modules belong to the same backend application.

Why is this useful ? Developers can work on different modules within the same project.

However, as the application grows, the codebase can become large and more difficult to understand.

Remember: A single codebase does not necessarily mean a single source file. It can contain thousands of files and many modules.

2. Tight Coupling :

Coupling refers to the degree of dependency between different components of a software application. When components depend heavily on one another, they are tightly coupled.

Example : Suppose your e-commerce application has three modules
Order , Payment , Notification

When a user places an order, the Order module calls the Payment module. After successful payment, the Order module calls the Notification module. Suppose The Payment module changes its function the Order module must be updated to provide the new argument . If other modules also call this function, they may need changes too. This is an example of how dependencies can make changes difficult.

Why is tight coupling a problem?

- Changes in one module can affect other modules
- Testing becomes more complicated
- Bugs can spread across different parts of the application
- Developers need to understand multiple modules before making certain changes

However, not every monolith is tightly coupled. A well-designed monolith can use modular architecture, interfaces, and dependency management to reduce coupling.

3. Shared Runtime : A runtime is the environment in which an application executes

For example

- a. [Node.js](#) is a runtime for javaScript application
- b. The Java Virtual Machine(JVM) runs Java applications
- c. Python has an interpreter that executes Python programs

In a traditional monolithic application, its modules usually run within the same application process.

Example : When this application starts, the different modules are loaded into the same Node.js application process. The modules can communicate through ordinary function calls. This function call doesn't require an HTTP request to another application.

Why is this Useful ? Communication within the same process is generally faster than communication over a network.

It also avoids some network-related problems such as

Network timeouts

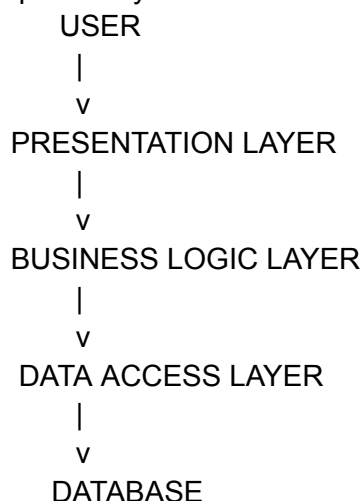
Connect failures

Dns resolution failures

Inter-service authentication

However, a shared runtime also means that a process-level failure can affect multiple modules

4. Layered Structure : A monolithic application can still have a well-organized internal structure. The application is divided into layers, with each layer having a particular responsibility.



Important: Layered architecture and monolithic architecture are not opposites. A monolithic application can be internally organized into several layers.

5. Limited Independent Scalability : Scalability refers to an application's ability to handle increasing workloads.

Suppose your e-commerce application receives 1,000 requests per second. Most requests are product searches. The search functionality may need more computing resources than the other modules. In a traditional monolith, you generally scale the entire application rather than scaling only the search module.

Each application instance contains all the modules. Even if only product search needs more capacity, adding another instance also runs the user, cart, order, and payment modules. This can lead to inefficient resource usage.

6. Single Unit Deployment : Deployment is the process of making an application available in an environment where users can access it. In a monolithic architecture, the application is typically packaged and deployed as one unit.

Suppose you make a change to the Notification module. Even though you changed only the Notification module, the whole application is redeployed. This can make deployments more time-consuming as the application grows.

Importance and Advantages of Monolithic Architecture

Developers still use it because it offers several practical advantages.

1. **Similarity :** A monolithic application is often easier to set up initially. You don't need to create multiple independent services or configure communication between them.

For example : to build a small e-commerce application , you can create a single backend with User routes, Product routes, Cart routes, Order routes , Payment routes.

You can start the entire backend with one command : `npm start`

You don't need to start five different backend services This makes development and initial deployment relatively straightforward.

2. **Cost effectiveness :** For small teams and applications , this can be particularly useful.
3. **Performance :** In a monolithic application, modules can communicate using direct function calls. In a microservices architecture, the equivalent operation might require an HTTP request to a separate Product service .Network communication introduces additional overhead. Therefore, for some workloads, communication within a monolith can be faster.
4. **Easier Debugging :** Suppose a user reports that placing an order produces an incorrect total. In a monolith, you can inspect the relevant code within one application. You can also trace function calls within the same process.

Challenges of Monolithic Architecture

1. **Long Deployment Cycles** : Suppose your application contains 20 modules. You made a small change to the product module. Since the application is deployed as one unit, the usual deployment process involves building and testing the complete application.

Also as the application grows, the following activities can take longer

- a. Compilation or bundling
- b. Automated testing
- c. Packaging
- d. Deployment
- e. Verification

2. **Risk of Downtime**: Suppose you need to deploy a new version of your e-commerce application. If your deployment process stops the existing application before starting the new version, users may temporarily be unable to access it. However, we can avoid this downtime using different deployment methods, which will be discussed shortly.

3. **Limited Scalability** : Suppose the Product module receives 10,000 requests per second, while the Payment module receives only 100 requests per second. In a traditional monolith, you generally scale the entire application. For example, you might increase the number of application instances from 2 to 10. Every instance runs all the modules. This means you're also running additional copies of modules that may not need more resources. This can increase infrastructure costs.

4. **Resource Consumption** : Imagine the Product module requires a lot of memory, but the Cart and User modules need very little. If you scale the entire application to handle the Product module's workload, you also allocate resources to the other modules. Some of those resources may remain underutilized. However, this is not always a problem. If all modules have similar resource requirements, scaling the entire application may be efficient.

5. **Limited Flexibility** : Suppose your application is written entirely in Java. Later, your team wants to use Python for a machine learning recommendation engine. In a monolithic application, integrating a different language may require additional infrastructure or a separate process. It is possible to build a monolith using multiple languages, but doing so may introduce complexity.

MICROSERVICES ARCHITECTURE : Microservices architecture is a software architecture in which an application is divided into multiple small, independent services.

Each service is responsible for a specific business capability. These services communicate with one another through network-based mechanisms such as HTTP APIs, REST APIs, gRPC, message queues, Event streams

Unlike a traditional monolith, each microservice can generally be developed, deployed, and scaled independently. Each service can have its own codebase, runtime, deployment process, and data store. The important part is that they have well-defined responsibilities and can be operated independently.

How Microservices Work ? Let's understand the complete working of microservices through an e-commerce purchase.

Suppose a user wants to buy a laptop.

1. The user sends a request : The user opens the application and clicks the *Buy Now* button. The frontend sends a request to the backend.
2. API Gateway receives the request : The API Gateway acts as a single entry point for external client requests.
It can :
Route requests to appropriate services , Perform authentication checks , Apply rate limits , Handle certain cross-cutting concerns.
3. Order Service processes the request .It may need to communicate with the Product Service to check whether the product is available. It sends a request to the Product Service. The Product Service checks its database and returns the product information.
4. Payment Service processes payment : After the order has been validated, the Order Service coordinates payment processing. It sends a request to the Payment Service. The Payment Service communicates with the payment provider to process the transaction. The Payment Service returns the result to the Order Service.
5. Order Service saves the order : After receiving the relevant results, the Order Service updates the order's status and stores the order information in its database.
6. Notification Service sends a notification : The Notification Service receives the event and sends an email or push notification. This can happen asynchronously, meaning the Order Service doesn't have to wait for the notification to finish before returning a response.

Characteristics of Microservices

- a. **Independent Services** : Each microservice handles a particular business capability.
- b. **Loose Coupling** : Loose coupling means that components have relatively few dependencies on one another.
For example, the Order Service may call: GET /products/501
It doesn't need to know how the Product Service internally stores its data. The Product Service could use PostgreSQL, MongoDB, or another database without requiring the Order Service to change, as long as the API contract remains compatible. This helps teams develop and maintain services independently.
However, loose coupling doesn't mean there are no dependencies.If the Order Service depends on the Product Service, a Product Service outage may still affect order processing
- c. **Independent Deployment** : You can build, test, and deploy the Cart Service without redeploying all the other services, provided the changes are compatible. This allows different teams to release their services at different times. These deployments don't necessarily have to happen together.
- d. **Independent Scalability** : Services can be scaled independently .This can improve resource utilization when workloads differ significantly.
- e. **Technology Diversity** : Microservices allow different services to use different technologies.It can be useful when different technologies are suited to different workloads. However, using too many languages and frameworks can increase maintenance and operational complexity.

f. Network communication : Microservices generally communicate through network protocols. Unlike a direct function call within a monolith, this request can fail because of: Network problems, TimeOuts , Service unavailability , Connection errors .Therefore, microservices need mechanisms such as timeouts, retries, and circuit breakers.

Components of Microservices Architecture

1. Microservices : Microservices are the individual services that make up the application.

2. API Gateway : An API Gateway is a component that acts as an entry point for client requests. Without an API Gateway, a frontend may need to communicate directly with many services. This means the frontend must know the addresses of multiple services. It may also need to handle authentication, errors, and other concerns for each service. An API Gateway simplifies this arrangement. The API Gateway forwards the request to the Product Service. The Product Service returns the product information. The API Gateway then returns the response to the frontend.

3. Service Registry and Service Discovery : A service registry maintains information about available service instances. A service discovery mechanism helps services find and communicate with those instances.

Example : The Order Service needs to communicate with the Product Service. But what happens if the Product Service runs on multiple instances? The Order Service needs to know which instances are available. When the Order Service needs the Product Service, it can obtain an available instance address through service discovery. Service discovery can be implemented through tools such as Kubernetes DNS or dedicated service discovery systems.

4. Load balancer: A load balancer distributes incoming requests across multiple service instances. This helps improve availability , Distribute workload, Avoid overloading one instance and Support horizontal scaling. Load balancing can happen at several levels, including at the API Gateway, the infrastructure layer, or within a service mesh.

5. Deployment and Infrastructure : Microservices need infrastructure to run. Common technologies include : Docker packages an application and its dependencies into a container image. For example, you could package the Product Service into a Docker image. Each microservice can have its own container image. Kubernetes is a container orchestration platform. It helps manage containerized applications. Suppose the Product Service needs three instances. You can configure Kubernetes to maintain three replicas. If one instance fails, Kubernetes can create another instance to restore the desired number. Docker and Kubernetes are common tools for microservices, but microservices do not strictly require either of them.

6. Event Bus or Message Broker: A message broker enables asynchronous communication between services. Why is asynchronous communication useful? It can Reduce direct dependencies between services, Allow services to process work independently. Help absorb temporary traffic spikes. Improve resilience when consumers are temporarily unavailable. However, it introduces challenges such as duplicate messages, delayed processing, and eventual consistency.

7. Database per Microservice : In microservices architecture, a common design principle is that each service owns its own data. Other services should access this

information through APIs or events rather than directly modifying another service's database. Each service can also choose a database that suits its needs.

8. Caching : Caching is also useful in microservices. Instead of querying the database every time, it can cache frequently accessed product information. This reduces repeated database queries and can improve response time. However, caching introduces the problem of keeping cached data up to date.

9. Fault Tolerance and Resilience :

Fault tolerance: The ability of a system to continue operating, possibly with reduced functionality, when some components fail.

Resilience: The ability of a system to recover from failures and continue providing its intended functionality. Some techniques used to achieve resilience are Timeouts, Retries, Circuit breakers, Fall backs, Health checks, Redundant service instances. Let's understand the circuit breaker.

Circuit Breaker : Imagine the Order Service repeatedly calls the Payment Service. The Payment Service is unavailable. If the Order Service continues making requests, it may waste resources and become overloaded. A circuit breaker can temporarily stop requests to the failing service. This prevents repeated calls to a service that is currently failing.

Challenges of Microservices : Microservices offer flexibility, but they also introduce additional complexity.

- 1. Service Communication :** In a monolith, one module can call another module directly. In microservices, services often communicate over a network. The request can fail because of network problems. Also, network communication generally has more overhead than a direct function call.
- 2. Network Latency :** Latency is the time taken for a request or operation to complete. Communication between services can increase latency. Developers can reduce this problem by : Combining related operations where appropriate. , Using asynchronous processing. , Caching frequently accessed data , Avoiding unnecessary service-to-service calls.
- 3. Data Consistency :** In a monolith, several modules may use the same database. They can sometimes perform related updates within one database transaction. While in microservices , each service may own its own database. These operations may involve three different services and databases. What happens if the payment succeeds but the inventory update fails? The system needs a way to handle the partial failure. One approach is the **Saga pattern**. A Saga breaks a long-running business transaction into smaller local transactions. If a later step fails, the system may execute compensating actions to undo or offset earlier steps. The actual workflow depends on the business requirements. This approach is more complex than a single database transaction.
- 4. Testing Complexity** In a monolithic application, developers can often run the entire application locally and test its modules together. With microservices, an application may consist of dozens of services. Testing all these services together can require considerable setup. Teams commonly use : Unit testing , Integration , Contract testing , End to End testing
- 5. Deployment Complexity :** Suppose an application has 30 microservices then each service may have its own Docker image , CI/CD pipeline ,

Configuration, Deployment process, Monitoring requirements. Managing all these independently requires automation and careful coordination. Tools such as Docker, Kubernetes, and CI/CD platforms can help. However, they also require expertise and maintenance.

6. **Monitoring and Logging** : In a monolith, a request may be handled within one process. In microservices, a single user request can pass through several services. If the request fails, you need to identify which service caused the failure. You may need to collect logs from multiple services. Distributed tracing helps track a request across services using a shared trace ID.

Feature	Monolithic Architecture	Microservices Architecture
Structure	One application containing multiple modules	Multiple independent services
Codebase	Usually one main codebase	Often multiple codebases
Deployment	Usually deployed as one unit	Services can be deployed independently
Communication	Usually direct function calls within a process	Network communication, APIs, or messaging
Coupling	Can become tightly coupled	Designed for loose coupling
Scaling	Usually scales the whole application	Individual services can be scaled
Technology	Often uses a common technology stack	Services can use different technologies
Testing	Can be simpler to test locally	Requires service and integration testing
Debugging	Often easier within one process	Requires distributed tracing and logs
Infrastructure	Usually simpler	Usually more complex
Initial cost	Often lower	Often higher
Deployment complexity	Relatively simple initially	More complex
Fault isolation	A process failure can affect the whole application	Service failures can be isolated, depending on dependencies

Data management	Often uses a shared database	Commonly uses database per service
Best suited for	Small and medium applications, simpler workloads	Systems needing independent deployment and scaling

Event Driven Architecture

Event-Driven Architecture is a software architecture in which different components communicate with each other by **producing and responding to events**.

An **event** is a notification that something has happened in a system.

For example :

A user places an order

A customer makes a payment

A user registers on a website

A product is added to a shopping cart

A delivery is completed

Whenever one of these things happens , an event is generated. Other components of the application can listen for these events and perform their respective tasks

Example : Amazon (Imagine you order a laptop from an online shopping application.)

Then

- a. The order is created
- b. You receive on order confirmation email
- c. The payment is processed
- d. The inventory is updated
- e. The delivery service is notified

All the tasks need to happen after you place an order

In a traditional application , the order service might directly call each of these services But in Event- Driven Architecture , the order service generated an event Order Placed Other services listen for this event and perform their tasks independently. The important thing is that the order service doesn't need to know how these other services work.It simply announces that an order has been placed.

How does Event Driven Architecture Work?

There are three main components in Event-Driven Architecture:

1. Event Producer
2. Event Broker
3. Event Consumer

Event Producer : An Event Producer is a component that generates or publishes an event when something happens in the application.

Suppose when a customer places an order , the order service generates an event

```
{  
  "event": "OrderPlaced",  
  "order_id": 101,  
  "product": "Laptop",  
  "quantity": 1  
}
```

This event contains information about what happened

Here

OrderPlaced is the event

Order_id identifies the order

Product identifies the purchased product

Quantity specifies how many products were ordered

Here the Order Service is the Event Producer because it generates this event.**In simple words:** An Event Producer is the component that announces that something has happened.

Event Broker : An Event Broker is a component that receives events from producers and distributes them to the appropriate consumers.

Think of it as a post office. Imagine you send a letter to someone. The post office receives the letter and delivers it to the correct destination. Similarly, an Event Broker receives events and makes them available to interested consumers.

Some popular event brokers are :

1. Apache Kafka
2. RabbitMQ
3. Amazon EventBridge
4. Google Cloud Pub/Sub

In simple words: An Event Broker acts as a middleman between producers and consumers. It helps components communicate without directly depending on one another.

Event Consumer : An Event Consumer is a component that receives and processes events. Consumers subscribe to events they are interested in. All these services are consumers because they receive and process the event. Notice that the Email Service doesn't need to know how the Order Service creates an order. It only needs to know what to do when it receives an **OrderPlaced** event.

In simple words: An Event Consumer is a component that listens for events and performs actions in response.

Complete working of Event-Driven Architecture (Let's put all three components together.)

Imagine a customer purchases a laptop

Step1 : An event occurs

The customer clicks the Place Order button , The order Service creates the order.

Step2 : The producer generated an event

The Order Service generates an **OrderPlaced** event. It publishes this event to the Event Broker.

Step3: The broker receives the event

The Event Broker receives the event and makes it available to the subscribed consumers.

Step4 : Consumers receive the event

Different consumers receive the event and perform their tasks.

Step5: Consumers process the event independently ,

Each consumer performs its own task without directly calling the other consumers. For example, the Email Service doesn't need to communicate with the Inventory Service. They both respond independently to the same event.

One important detail: these operations may happen asynchronously, meaning the Order Service doesn't have to wait for every consumer to finish before continuing. Also, the order of processing isn't necessarily guaranteed unless the system is specifically designed to enforce it

What is the difference between synchronous and asynchronous communication?

Synchronous communication : In synchronous communication, one service sends a request to another service and waits for its response. Suppose the Order Service needs to call the Payment Service. The Order Service waits until the Payment Service responds. If the Payment Service is slow, the Order Service may also have to wait.

Asynchronous communication: In asynchronous communication, a service sends a message or publishes an event without waiting for every consumer to finish processing it. The Order Service publishes the event and can continue its work. The consumers process the event independently.

The Key difference here is

Synchronous: The sender waits for a response.

Asynchronous: The sender doesn't have to wait for the consumer to finish processing .

Note that Event-Driven Architecture commonly uses asynchronous communication, but not every event-driven system is necessarily asynchronous.

Concept	Explanation
Event	A record of something that has happened in the system.
Producer	Generates and publishes events.
Broker	Receives and distributes events.
Consumer	Receives and processes events.
Topic	A named channel to which events are published and from which consumers can receive them.
Queue	A mechanism for storing messages until consumers process them.
Event Handler	The code that executes when a consumer receives an event.
Event Schema	Defines the structure and format of an event.

Important: Event-Driven Architecture and microservices are not the same thing. You can implement event-driven communication within a monolithic application. Similarly, microservices can communicate using synchronous REST APIs instead of events.

Advantages of Event Driven Architecture

1. **Loose Coupling** : Producers don't need to know the internal implementation of consumers.
2. **Scalability** : Different Consumers can be scaled independently .For example, if the Email Service receives a large number of events, you can run more instances of the Email Service without scaling the entire application.
3. **Asynchronous Processing** : Consumers can process events without blocking the producer. This is useful for tasks such as sending emails, generating reports, and updating analytics.
4. **Extensibility** : You can add new consumers without making significant changes to existing producers. For example, you could add an Analytics Service that listens to `OrderPlaced` events to calculate sales statistics.

Disadvantages of Event-Driven Architecture:

1. **Complexity** : You need to manage event brokers, subscriptions, event schemas, and message processing
2. **Difficult Debugging** : Since multiple services process events independently, tracking a request across the system can be challenging.

3. **Eventual consistency** : Different services may update their data at different times. For example, an order may be created before the inventory service updates the stock. For a short period, the order and inventory services may show different information.
4. **Duplicate Events**: A broker may deliver an event more than once. Consumers should be designed to handle duplicate events safely when necessary.
5. **Error Handling** : If a consumer fails to process an event, the system needs a way to retry it or handle it separately.

The Most important things to remember :

In a traditional request-response architecture, one service often needs to contact another service directly to ask it to perform an operation.

In Event-Driven Architecture, a service announces that something has happened. Other interested services decide how to respond.

The producer announces what happened; the consumers decide what to do about it. Once you understand this distinction, the rest of Event-Driven Architecture becomes much easier to understand.

Scalability : Scalability is the ability of a system to handle an increase in workload by adding resources without significantly affecting its performance.

Imagine you develop an e-commerce application

Initially, only 100 users use your application. One server is sufficient to handle all their requests. But after your application becomes popular, 10,000 users start accessing it simultaneously. You need a way to handle the increased number of users. This is where scalability comes in. A scalable system can handle increasing traffic by adding resources.

Types of Scalability :

1. Vertical Scaling (Scaling up) : Vertical scaling means **increasing the resources of an existing server** But there will be a limit to how powerful a single machine can be.
2. Horizontal Scaling (Scaling out) : Horizontal scaling means **adding more servers to distribute the workload**. Suppose your application initially runs on one server. When traffic increases, instead of upgrading that server, you add more servers. But this will be more complex to implement.

- Upgrade your existing server with more CPU and RAM (vertical scaling).
- Add more servers and distribute user requests among them (horizontal scaling).

For large applications, horizontal scaling is commonly used because it allows the application to grow by adding machines.

Availability : Availability is the ability of a system to remain operational and accessible when users need it.

How can we achieve availability ?

1. **Redundancy** : Redundancy means having additional components available to take over if a component fails. Ex : If Server 1 fails, the load balancer can send requests to Server 2. The application can continue operating.
2. **Failover** : Failover is the process of switching to a backup component when the primary component fails. Ex : If the primary database fails, a replica may be promoted to become the new primary. The exact behavior depends on the database and its configuration.
3. **Health Checks** : A load balancer can periodically check whether servers are functioning correctly. If Server 2 fails its health checks, the load balancer stops sending it new requests.
4. **Replication** : Database replication maintains copies of data on multiple database servers. If one database server becomes unavailable, another may be able to serve requests. However, replication alone does not guarantee automatic failover or zero data loss.

Scalability vs Availability : These two concepts are related, but they solve different problems.

Scalability	Availability
Handles increasing workloads	Keeps the system operational
Focuses on capacity	Focuses on uptime
Adds resources to handle more requests	Uses redundancy and failover to handle failures
Example: Adding more servers during a traffic spike	Example: Redirecting traffic when a server crashes

CAP Theorem : The CAP theorem is one of the fundamental concepts in distributed systems. Before understanding it, let's understand what a distributed system is.

What is a distributed system? A distributed system is a collection of independent computers that communicate over a network and work together as one system. These components communicate over a network to provide a single application to users.

Example : Now, imagine your database is replicated across two servers. Both databases contain copies of the same data. But what happens if the network connection between them fails? This is where the CAP theorem becomes important.

CAP Stands C- consistency , A - Availability , P - Partition Tolerance

Consistency : Consistency ensures that users do not see outdated data when the system promises strong consistency.

Availability : In the CAP theorem, availability means that every request received by a non-failing node gets a response. The response does not necessarily have to contain the latest data.

Ex: Database A and Database B are connected. Now, the network connection between them fails . A user sends a read request to Database B. If DataBase B is configured to continue serving requests , it can return its locally stored data. Even if that data is outdated , it is still responding . That is availability in the CAP sense

Partition Tolerance : A network partition occurs when some nodes in a distributed system cannot communicate with each other because of a network failure. Ex : Both databases are still running, but they cannot communicate. A distributed system is partition-tolerant if it continues operating despite such a communication failure, according to its design. In real distributed systems, network failures can happen. Therefore, systems generally need to be designed to tolerate partitions.

Understanding The CAP theorem : The CAP theorem states that when a network partition occurs, a distributed system cannot guarantee both strong consistency and availability for every request.

It does not mean that a system can only ever have two of the three properties under all circumstances. Rather, when a partition occurs, the system must make a choice between consistency and availability.

Feature	CP	AP
Full form	Consistency and Partition Tolerance	Availability and Partition Tolerance
During a partition	May reject or delay requests	Continues responding to requests
Data	Prioritizes strong consistency	May return stale data
Availability	Some requests may fail	Prioritizes successful responses
Example use case	Banking transactions	Social media feeds
Main trade-off	Availability	Immediate consistency

What about CA? CA stands for Consistency and Availability. A system can provide consistency and availability when there is no network partition. However, a distributed system cannot guarantee both during a partition while also tolerating that partition. Therefore, CA is not a viable guarantee for a distributed system that must continue operating through network partitions.

Load Balancing : Load balancing is the process of distributing incoming network traffic across multiple servers to prevent any one server from becoming overloaded.

The load balancer distributes requests across the three servers.

This helps :

- a. Prevent individual servers from becoming overloaded
- b. Improve response times
- c. Increase the application's capacity
- d. Improve availability when combined with health checks and failover

How does a load balancer work ?

Suppose 6 users send requests to your application . A load balancer receives the requests and forwards them to available servers. The load balancer decides which server should receive each request based on its configured algorithm.

Types of load balancing algorithms:

1. **Round Robin :** Round Robin distributes requests in a circular order between the servers

Adv :

- a. Simple to implement and distributes requests evenly when servers have similar capacities and requests have similar costs

DisAdv :

- a. Does not consider the current server load and can be inefficient if requests require different amounts of processing.

2. **Weighted Round Robin :** Weighted Round Robin is similar to Round Robin, but servers receive different numbers of requests based on their assigned weights. This is useful when servers have different capacities.

3. **Least Connections :** The load balancer sends a new request to the server with the fewest active connections. Suppose Server A has 10 active connections, Server B has 3 , Server C has 7 active connections . So the next request is sent to Server B . This can be useful when requests take different amounts of time to complete.

4. **IP Hash :** The load balancer uses a hash of the client's IP address to select a server. Something like $(i) \% \text{total servers}$ or for example

Client A ---> Server 1

Client B ---> Server 2

Client C ---> Server 3

If the same client sends another request, the hash may direct it to the same server, provided the server pool and configuration have not changed. This can help with session persistence, although it does not guarantee that sessions will survive server failures or configuration changes.

Layer 4 vs layer 7 Load Balancing : Load balancers can operate at different layers of the network stack.

Feature	Layer 4 Load Balancer	Layer 7 Load Balancer
OSI layer	Transport	Application
Works with	TCP/UDP information	HTTP/HTTPS and application information
Routing	IP address and port	URL path, hostname, headers, cookies
Complexity	Generally simpler	More complex
Example	Route TCP traffic to port 8080	Route <code>/api</code> to one service and <code>/images</code> to another

A load balancer with health checks can detect that Server 2 is unhealthy and stop sending it new requests. The other servers continue handling traffic. This improves availability, provided they have sufficient capacity. However, the load balancer itself can also become a single point of failure. Production systems often use redundant load balancers or managed load-balancing services.

Proxy Servers : A proxy server is an intermediary that forwards requests between a client and another server.

Normally, when you access a website, your browser sends a request directly to the website's server. Like this Client -----> Web Server

HTTP Request

With a proxy server , the request passes through an intermediary like

Client → Proxy Server → Web server

The proxy receives the request and forwards it to another server. There are 2 important types of proxies :

- a. Forward Proxy
- b. Reverse Proxy

Forward Proxy : A forward proxy acts on behalf of the client. It sits between the client and the destination server.

Ex : Suppose you are using a computer in a college library

The college may configure a forward proxy for internet access. When you try to access a website , your request first goes to the proxy server. The proxy then forwards the request to the website. The response comes back through the proxy.

Your computer → Request → Forward Proxy → Internet → Google Server → Response
→ forward proxy → your computer

Why do we use a forward proxy ?

1. Privacy : A forward proxy can hide the client's IP address from the destination server. However, the proxy itself may still know the client's IP address.

2. **Access Control** : Organizations can use forward proxies to control which websites their employees or students can access. For example, a college may block certain websites on its network.
3. **Caching** : A proxy can cache frequently requested content. If multiple clients request the same cacheable resource, the proxy may serve it from its cache instead of retrieving it repeatedly from the original server.
4. **Monitoring** : Organizations can use proxies to monitor or log network traffic, subject to their policies and applicable laws.

Reverse Proxy : A reverse proxy acts on behalf of the server. It sits between clients and the backend servers. Unlike a forward proxy, a reverse proxy is generally managed by the organization operating the website.

Suppose you visit an e-commerce website. You send a request to the website's public domain. You do not need to know which backend server will handle your request. The reverse proxy receives your request and forwards it to an appropriate backend server.

How does a reverse proxy work ? Imagine you have an e-commerce application with three backend servers. When a user requests a product page, the reverse proxy can forward the request to one of the available servers. The response is then returned to the client through the reverse proxy. The client communicates with the reverse proxy rather than directly with the backend servers.

Why do we use a reverse proxy ?

1. **Load balancing** : A reverse proxy can distribute incoming requests among multiple backend servers.
2. **Security** : It can hide the backend servers' internal addresses and act as a controlled entry point into the application.
3. **SSL / TLS Termination** : A reverse proxy can handle HTTPS encryption and decryption.
4. **Caching** : A reverse proxy can cache responses from backend servers.
5. **Routing** : A reverse proxy can route requests to different backend services based on the hostname or URL path. This is particularly useful in applications built with microservices.

Forward proxy:

Client ---> Proxy ---> Internet ---> Server



It represents the client.

Reverse proxy:

Client ---> Internet ---> Proxy ---> Backend Server



It represents the server.

Think of it this way:

- A forward proxy is concerned with outgoing requests from clients.
- A reverse proxy is concerned with incoming requests to servers.

Feature	Forward Proxy	Reverse Proxy
Represents	Client	Server
Location	Between clients and the internet	Between clients and backend servers
Main purpose	Controls or forwards client requests	Manages incoming requests to servers
Hides	Client's IP address from the destination	Backend server details from clients
Used by	Clients, organizations, schools	Websites and application providers
Load balancing	Not its usual purpose	Commonly used
Caching	Can cache responses for clients	Can cache responses for users
Security	Can enforce outbound access policies	Can protect and control access to backend servers
Example	College internet proxy	Nginx in front of application servers

Don't forget to revise these

1. Caching : Caching means storing frequently accessed data temporarily so that it can be retrieved faster.
2. Database Replication : Database replication means maintaining copies of a database on multiple servers.
3. Database Sharding : Sharding means splitting a database into smaller pieces called shards and distributing them across multiple database servers.
4. Data Partitioning : Data partitioning means dividing a large dataset into smaller parts (on same server)
5. Content Delivery Network (CDN) : A CDN is a geographically distributed network of servers that delivers content from locations closer to users.
6. Queueing Systems : A queueing system stores tasks until they can be processed.

Quick revision notes

Concept	Meaning	Main purpose
Scalability	Ability to handle increasing workloads	Handle more users and requests
Vertical scaling	Increasing the resources of one server	Increase the capacity of a machine
Horizontal scaling	Adding more servers	Distribute workload
Availability	Ability to remain operational	Minimize downtime
CAP theorem	Consistency and availability cannot both be guaranteed during a network partition in a partition-tolerant distributed system	Understand distributed-system trade-offs
Load balancer	Distributes requests across servers	Prevent overload and improve availability
Forward proxy	Represents the client	Control and forward outgoing requests
Reverse proxy	Represents the server	Manage incoming requests
Caching	Temporarily stores frequently used data	Reduce latency and backend load
Replication	Creates copies of data	Improve read capacity and availability
Sharding	Splits data across multiple databases	Scale database storage and processing
Microservices	Separates an application into independent services	Scale and deploy services independently
CDN	Serves cached content from distributed locations	Reduce latency and origin traffic
Queue	Stores tasks for later processing	Handle spikes and process tasks asynchronously

